

Agentic RFP Evaluation and Supplier Ranking

Prepared by kvl.narasimharao

This report presents a working application that compares supplier responses to a request for proposal. It explains how proposal PDFs are read, how criterion scores are checked, how Python calculates peer results and final ranks, and what the four-supplier demonstration produced. The repository includes the running application, sample proposals, saved results, tests, and screenshots.

Project purpose

A procurement team must review several supplier proposals against the same requirements. Manual reading makes the comparison slow and can lead to inconsistent scoring. This application produces an evidence-linked scorecard for each proposal and applies one published set of arithmetic and ranking rules to every supplier.

The language model judges proposal content and supplies criterion scores, reasons, and evidence. Python validates that response, calculates every comparison measure, and assigns the final order. The model does not set benchmarks or ranks.

What the application does

- Loads active criteria and weights from SQLite and displays them in the Criteria tab.
- Accepts at least two supplier PDFs with supplier name, submission date, and historical experience rating.
- Evaluates each PDF independently against the criteria active at the start of the batch.
- Corrects missing, malformed, duplicate, and out-of-range model results and records warnings.
- Calculates weighted scores, peer benchmarks, gaps, relative percentages, PPI, and final ranks in Python.
- Saves the full batch under one run ID and offers leaderboard, scorecards, and JSON download.

Submission requirements and included files

The source brief is Agentic_RFP_Evaluation_Mini_Project.pdf, section 10, Submission Requirements. Every requested item is present in the public repository or at the live application link.

Required item	Included deliverable
Complete source code	streamlit_app.py, rfp/, scripts/, tests/, and requirements.txt in the public repository
SQLite setup	scripts/seed_db.py and rfp/database.py create and seed five sample criteria
Four supplier PDFs	data/proposals/apex_systems.pdf, brightpath_tech.pdf, nexaworks.pdf, orbit_digital.pdf
Public deployed app	https://rfp-evaluation-kvlnarasimharao.streamlit.app/
README	README.md contains setup, architecture, formulas, assumptions, and screenshots
Completed JSON export	data/live_run.json stores one completed four-supplier RFP run
Short demonstration	Sample run shows a successful Gemini result and a separate validation example

Source code structure

Path	Responsibility
streamlit_app.py	Upload form, criteria editor, leaderboard, detailed scorecards, saved runs, export
rfp/evaluation.py	PDF extraction, dynamic prompt, model request, response normalization
rfp/scoring.py	Weighted arithmetic, peer comparison, deterministic ranking
rfp/workflow.py	Ordered batch evaluation and persistence
rfp/database.py	SQLite schema, criteria operations, completed run records
scripts/ and tests/	Sample generation, database seed, live run, document build, automated checks

System workflow

The user uploads the proposals and enters metadata. The orchestrator reloads active criteria, reads text from each PDF, and builds a prompt from those database rows. Gemini or Ollama Cloud returns a JSON assessment for one supplier at a time. The validator creates a complete, bounded scorecard even when fields are missing or malformed. Once all suppliers have scorecards, Python calculates the peer measures and sorts the batch. The complete result is committed to SQLite and shown in Streamlit.

Stage	Input	Output
Setup	SQLite criteria	Active criteria with total weight of 100%
Evaluate	PDF text and active criteria	Scores, reasons, evidence, risks, summary
Validate	Model JSON	Complete scorecards and warnings
Compare	Validated supplier scores	Benchmarks, gaps, relative results, PPI
Finish	Peer results and metadata	Ranks, saved run, leaderboard, JSON export

Evaluation criteria

Criteria are stored in SQLite rather than fixed in the prompt. The editor allows activation, deactivation, and weight changes. A batch cannot start unless active weights total 100%. Each criterion uses a maximum score of 10 in the included demonstration.

Criterion	Weight	What is inspected
Technical Capability	30%	Architecture, integrations, scalability, technical fit
Implementation Plan	20%	Timeline, milestones, staffing, risk plan
Commercial Value	20%	Pricing clarity, total cost, assumptions
Security & Compliance	20%	Controls, certifications, privacy, auditability
Support & Experience	10%	Support model, similar projects, references

Scoring and tie breaks

For each criterion, the absolute contribution is the validated score divided by its maximum score, multiplied by the criterion weight. The absolute score is the sum of those contributions. The benchmark is the highest validated score for that criterion among suppliers in the batch. A supplier's gap is its score minus the benchmark, so a benchmark leader has a gap of zero.

Relative performance is supplier score divided by benchmark, multiplied by 100. When every supplier scores zero on a criterion, the benchmark is zero and relative performance is set to 100% for all suppliers on that criterion. PPI is the sum of each relative percentage multiplied by its criterion weight divided by 100. It is calculated before display rounding is applied.

Final sorting is fixed: higher PPI first, then earlier submission date, then higher historical experience rating, then supplier name in ascending alphabetical order. Sequential ranks start at 1 after sorting. Experience affects a tie break only; it does not change criterion or absolute scores.

Worked calculation for Apex Systems

Apex received scores of 9, 9, 7, 9, and 8 out of 10 for the five criteria. Its absolute score is $(9/10 \times 30) + (9/10 \times 20) + (7/10 \times 20) + (9/10 \times 20) + (8/10 \times 10) = 85$. The benchmark for each criterion in the four-supplier run was 9. Apex therefore had relative results of 100%, 100%, 77.7778%, 100%, and 88.8889%. Applying the five weights gives a PPI of 94.4445 after display rounding.

Model response validation

The evaluator requests valid JSON with one entry per active criterion, including criterion ID, score, explanation, and supporting evidence. The validator uses the database criterion IDs and maximum scores as the source of truth. It applies the following rules before any arithmetic:

- Invalid JSON or a non-object response becomes an empty assessment; missing criteria receive zero.
- Unknown and duplicate criterion IDs are ignored. Missing expected criteria receive zero.
- Non-numeric or non-finite scores become zero. Scores outside the range are clipped.
- Missing explanations receive a placeholder. Missing evidence is flagged in the warnings.
- A malformed risks field becomes an empty list. Every correction is recorded with the supplier name.

Supplier names must be unique within a batch. Experience ratings must be between 0 and 10. Submission dates are checked before ranking. The application rejects PDFs that contain too little selectable text.

Stored data and output

SQLite has three tables: evaluation_criteria, rfp_runs, and supplier_results. The first stores the editable rubric. The run table stores the creation time, criteria snapshot, warnings, and complete result JSON. The supplier table stores each supplier's metadata, absolute score, PPI, rank, and scorecard. One RFP_RUN_ID connects all suppliers in a batch. The API key is not written to these tables or to the JSON export.

The interface has Evaluate, Criteria, Saved runs, and Sample run tabs. A completed run displays the ranked leaderboard, every criterion's score and evidence, peer comparisons, warnings, and a JSON download. The Sample run tab includes the completed Gemini result and a separate validation example.

Demonstration data

Supplier	PDF file	Intended difference
Apex Systems	apex_systems.pdf	Strong design and security; higher price
BrightPath Tech	brightpath_tech.pdf	Lowest price and fast timeline; weak compliance detail
NexaWorks	nexaworks.pdf	Balanced cost; strongest plan and support
Orbit Digital	orbit_digital.pdf	Strong references; vague integration scope

Four fictional two-page proposals were prepared for the same university procurement request. They differ in architecture, implementation timing, cost, security detail, support, and references. Gemini 2.5 Flash evaluated all four in a completed batch. The table reports Python's final calculations; scores in the five criterion columns are out of 10.

Rank	Supplier	Tech	Plan	Value	Security	Support	Absolute	PPI
1	Apex Systems	9	9	7	9	8	85.0	94.4445
2	NexaWorks	9	9	8	6	9	82.0	91.1111
3	Orbit Digital	6	7	6	7	9	67.0	74.4445
4	BrightPath Tech	4	6	9	1	3	47.0	52.2222

Apex Systems ranked first with strong technical and security detail. NexaWorks followed with a strong plan and support offer. Orbit Digital had substantial references but less detail on ERP integration. BrightPath Tech had the lowest price but limited security detail. The model returned a malformed risks field for BrightPath; validation replaced it with an empty list and retained its valid criterion scores.

The saved run ID is d3bee8e3-cb68-4183-8d1a-9d9429742653. The complete result is in data/live_run.json. A separate data/sample_run.json deliberately includes an out-of-range score so the correction path can be demonstrated without a model call.

Testing and verification

Check	Result
Response normalization	Missing results, duplicate IDs, invalid values, and clipping are covered.
Scoring and ranking	Weighted score, peer measures, zero benchmark, and every tie break are covered.
Persistence	A complete two-supplier run is saved and loaded from SQLite.
Application	The live app opens; Sample run shows four ranks, scorecards, warnings, and JSON download.

All four automated tests passed. The four proposal PDFs were evaluated in one Gemini batch, and the local interface was checked for criteria editing, input, sample results, scorecards, and export. The public Streamlit app was opened and its saved sample run was checked.

Demonstration sequence

Successful run: open the live app and select Sample run, then Live Gemini evaluation. The leaderboard shows four final ranks. Expand Apex Systems to inspect the criterion explanations, supporting evidence, benchmark, gap, and relative percentages. Use Download complete result as JSON to export the saved run.

Validation case: in Sample run, select Validation example. Its deliberately out-of-range criterion score is clipped to zero before scoring, and the warning is shown in Run details. The completed Gemini example also displays a malformed risks-field warning for BrightPath Tech. These cases demonstrate that model output is checked before ranking.

How to run and present the project

Public repository: <https://github.com/kvlnarasimharao/agentric-rfp-evaluation>

Live application: <https://rfp-evaluation-kvlnarasimharao.streamlit.app/>

In the live app, open Sample run to review the completed comparison, then open Criteria to view the active weights. To evaluate new proposals, use Evaluate, upload at least two text-based PDFs, fill supplier metadata, and select Evaluate suppliers. The hosted Gemini key is held in Streamlit secrets, so no key entry is needed for that deployment.

For a local run, clone the repository, install packages from requirements.txt, run python scripts/seed_db.py, then run streamlit run streamlit_app.py. Set GEMINI_API_KEY locally to evaluate a new batch. The sample run is viewable without a key. Run python -m unittest discover -s tests -v to repeat the automated checks.

Operating conditions

PDFs must contain selectable text; scanned images require text recognition before upload. The model's judgment and cited evidence should be reviewed for procurement use. The public cloud deployment uses local SQLite storage that may reset when the app instance restarts, so completed results should be downloaded as JSON when a durable record is needed.

References

Source brief, Agentic_RFP_Evaluation_Mini_Project.pdf, sections 1 to 10

Google AI for Developers, Gemini generateContent API: <https://ai.google.dev/api/generate-content>

Streamlit, Deploy your app on Community Cloud: <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy>

Streamlit, Secrets management: <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/secrets-management>

Application screenshots

Completed four-supplier evaluation in the Sample run tab.

RFP Supplier Evaluation

Compare supplier proposals using document evidence and transparent ranking rules.

Evaluate Criteria Saved runs Sample run

Example

Live Gemini evaluation

Both examples use the four fictional supplier proposals.

Leaderboard

Rank	Supplier	Absolute score / 100	PPI / 100	Submitted	Experience / 10
1	Apex Systems	85	94.4445	2026-09-15	8.5
2	NexaWorks	82	91.1111	2026-09-16	8
3	Orbit Digital	67	74.4445	2026-09-13	9.5
4	BrightPath Tech	47	52.2222	2026-09-14	4

Order: higher PPI, earlier submission, higher experience, then supplier name A to Z.

Active criteria and weights loaded from SQLite.

RFP Supplier Evaluation

Compare supplier proposals using document evidence and transparent ranking rules.

Evaluate Criteria Saved runs Sample run

Evaluation criteria

criterion_id	name	description	weight	max_score	Active
1	Technical Capability	Architecture, integrations, scalability, technical fit	30	10	☒
2	Implementation Plan	Timeline, milestones, staffing, risk plan	20	10	☒
3	Commercial Value	Pricing clarity, total cost, assumptions	20	10	☒
4	Security & Compliance	Controls, certifications, privacy, auditability	20	10	☒
5	Support & Experience	Support model, similar projects, references	10	10	☒

Current active weight: 100%

Save criteria